AI/ML Track

Interview Preparation Series • Professional Edition

ML Fundamentals

01

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

1 Machine Learning Fundamentals

> The art of making computers learn from experience so we don't have to hand-code every rule — the foundation of every intelligent system at FAANG.

1.1 Overview --- What It Is

Machine Learning (ML) is a subfield of Artificial Intelligence where systems learn patterns from data and improve their performance on tasks **without being explicitly programmed for each case**.

The Classic Definition

Arthur Samuel (1959): "The field of study that gives computers the ability to learn without being explicitly programmed."

Tom Mitchell (1997): "A computer program is said to learn from experience **E** with respect to task **T** and performance measure **P** if its performance at **T**, as measured by **P**, improves with experience **E**."

Symbol	Meaning	Example (spam filter)
E	Experience	Labeled emails (spam/not spam)
T	Task	Classify incoming emails
P	Performance	Accuracy on new emails

The Core Shift in Thinking

```
Traditional Programming:
  Rules + Data → Computer → Output

Machine Learning:
  Data + Output (labels) → Computer → Rules (model)
```

The model IS the learned rules — a function `f(x) → y` where:

- `x` = input features (pixel values, word counts, transaction amounts)
- `y` = output (label, score, next token)
- `f` = learned mapping (parameters/weights trained from data)

The Four Learning Paradigms

Paradigm	Has Labels?	Signal	Example
Supervised	Yes, explicit	Direct feedback on every prediction	Image classification, spam detection
Unsupervised	No	Structure in data itself	Customer clustering, anomaly detection
Self-Supervised	No explicit, auto-generated	Predict part of input from rest	GPT (predict next token), BERT (fill mask)
Reinforcement	No labels, rewards	Sparse reward signal after actions	AlphaGo, game-playing agents, RLHF

Supervised Learning — Deep Dive The model learns a mapping from inputs to outputs using labeled training data.

Classification — output is a discrete category:

- Binary: spam/not spam, fraud/legit

- Multi-class: digit recognition (0–9), ImageNet (1000 classes)
- Multi-label: a photo can be tagged ”dog”, ”outdoor”, ”sunny” simultaneously

Regression — output is a continuous value:

- House price prediction
- Stock return forecasting
- Temperature prediction

```source
# Supervised learning conceptual loop
for epoch in range(num_epochs):
    for x_batch, y_batch in dataloader:
        y_pred = model(x_batch)          # Forward pass
        loss = loss_fn(y_pred, y_batch)  # Compare to ground truth
        loss.backward()                  # Compute gradients
        optimizer.step()                 # Update weights
```

Unsupervised Learning — Deep Dive No labels. The model discovers inherent structure.

- **Clustering:** K-Means, DBSCAN, Hierarchical — group similar points
- **Dimensionality Reduction:** PCA, t-SNE, UMAP — compress features while preserving structure
- **Density Estimation:** GMM, KDE — model the data distribution
- **Anomaly Detection:** points far from learned distribution are anomalies

Self-Supervised Learning — Deep Dive The model creates its own supervision signal from unlabeled data. The *pretext task* generates pseudo-labels automatically.

Examples:

- **Language Modeling (GPT):** predict next word → learns world knowledge
- **Masked Language Model (BERT):** mask 15% of tokens, predict them → learns context
- **SimCLR / CLIP:** learn representations by pulling augmented views of same image close, pushing different images apart

Why it matters for FAANG: Enables training on petabytes of unlabeled web data. Foundation models are built this way.

Reinforcement Learning — Deep Dive An **agent** interacts with an **environment**, taking **actions** to maximize cumulative **reward**.

```
Agent → Action → Environment → State + Reward → Agent
```

Key terms:

- **Policy π:** mapping from state to action
- **Value function V(s):** expected future reward from state s
- **Q-function Q(s,a):** expected future reward from state s taking action a
- **Exploration vs Exploitation:** must try new things but also use what works

FAANG usage: recommendation system ranking, ad bidding, RLHF for LLM alignment (ChatGPT).

1.2 Why It Exists

The Scaling Argument --- Why Rules Don't Work

Humans cannot write explicit rules for:

1. **Scale:** Google processes 8.5 billion searches/day. No team can hand-code rules for every query.

2. **Complexity:** Face recognition requires capturing ~50 million pixel interactions. Rules are intractable.
3. **Adaptation:** User preferences shift constantly. Hardcoded rules go stale.
4. **Unknown patterns:** Fraud patterns are novel by design. ML detects statistical anomalies without knowing what fraud looks like in advance.

The Statistical Argument

ML is applied statistics at scale. Given enough data, a model can approximate any function (Universal Approximation Theorem for neural networks). We are essentially doing **function approximation** — finding the best `f` from a hypothesis space `H` that maps inputs to outputs.

The Economic Argument

Labor costs for labeling data (crowdsourced) are vastly cheaper than engineering every rule. A single model can replace thousands of if-else branches and outperform them.

1.3 Why FAANG Cares

Each company has multi-billion dollar ML bets. Interviewers want engineers who understand ML at a system level, not just as a black box.

Company	ML Use Cases	Why Fundamentals Matter
Google	Search ranking (BERT/MUM), ads CTR prediction, Gmail spam, Maps ETA, Translate, YouTube recommendations	Every product is powered by models; engineers need to understand model quality signals and deployment
Meta	News Feed ranking, ad targeting, content moderation, Reels recommendations, Instagram explore	Billions of predictions/second; bias-variance and calibration directly impact revenue
Amazon	Product recommendations (”customers also bought”), Alexa NLU, fraud detection, demand forecasting, delivery ETA	Wrong recommendations = lost revenue; demand forecasting affects billion-dollar inventory
Apple	Siri NLP, Face ID (on-device), autocorrect, Health app anomaly detection, photo classification	On-device ML with strict memory/latency; privacy constraints change the training paradigm
Microsoft	Copilot (GitHub/Office), Azure ML platform, Bing search, Xbox matchmaking	Platform-level ML tooling; Azure competes with GCP/AWS on ML infra
Uber	Surge pricing ML, ETA prediction, fraud detection, driver-rider matching, Eats recommendations	Real-time predictions under strict latency SLAs; poor calibration = driver/rider unhappiness
Netflix	Content recommendations (saves $1B/year per their claim), thumbnail personalization, encoding quality prediction	A/B testing culture; every model change is measured against watch time and retention
OpenAI	Foundation model training (GPT), RLHF alignment, safety classifiers, API latency optimization	Pushing the boundary of what ML fundamentals (loss surfaces, optimization) look like at 100B+ parameters

Interview Takeaway: Every FAANG interviewer expects you to connect ML concepts to business metrics — not just formula recall.

1.4 Core Concepts

The End-to-End ML Lifecycle

```
Problem Definition → Data Collection → EDA → Feature Engineering →
Model Selection → Training → Evaluation → Hyperparameter Tuning →
Deployment → Monitoring → Retraining
```

Each phase has failure modes that interviews love to probe:

Phase	Common Failure	Interview Signal
Problem Definition	Wrong metric (optimizing accuracy on imbalanced data)	Do you challenge requirements?
Data Collection	Selection bias, survivorship bias	Do you question data provenance?
Feature Engineering	Data leakage	Do you know why leakage is catastrophic?
Model Selection	Too complex for data size	Do you consider bias-variance?
Evaluation	Only using test set for tuning	Do you understand train/val/test discipline?
Deployment	Training-serving skew	Do you think about production?
Monitoring	No data drift detection	Do you think about model decay?

Train / Validation / Test Split

The Golden Rule: The test set is touched EXACTLY ONCE — after all decisions are made.

```
All Data
├── Training Set (60–70%)    ← Model learns parameters
├── Validation Set (15–20%)  ← Tune hyperparameters, model selection
└── Test Set (15–20%)        ← Final, unbiased performance estimate
```

Why three sets?

- Train on train set
- Use val set to pick best hyperparameters (learning rate, depth, regularization)
- If you use test set for tuning, you've overfit to the test set — it's no longer unbiased

```source
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
# Result: 70% train, 15% val, 15% test
```

Interview Gotcha: What if your dataset has only 1000 samples? A 700/150/150 split leaves too little for robust test estimation → use cross-validation.

Cross-Validation

K-Fold CV: split training data into K folds, train on K-1, validate on 1, rotate.

```
5-Fold CV:
Fold 1: [VAL] [TRN] [TRN] [TRN] [TRN]  → score_1
Fold 2: [TRN] [VAL] [TRN] [TRN] [TRN]  → score_2
Fold 3: [TRN] [TRN] [VAL] [TRN] [TRN]  → score_3
Fold 4: [TRN] [TRN] [TRN] [VAL] [TRN]  → score_4
Fold 5: [TRN] [TRN] [TRN] [TRN] [VAL]  → score_5

Final CV Score = mean(score_1..5) ± std(score_1..5)
```

Variant	Use Case
Stratified K-Fold	Classification with class imbalance — preserves class ratio in each fold
Time-Series Split	Temporal data — always train on past, validate on future (no leakage)
Leave-One-Out (LOO)	Tiny datasets — each sample is the validation set once (expensive)

Variant	Use Case
Nested CV	Hyperparameter tuning without test set contamination (outer loop = eval, inner loop = tuning)

```source
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=skf, scoring='roc_auc')
print(f"AUC: {scores.mean():.3f} ± {scores.std():.3f}")
```

The Bias--Variance Tradeoff

The fundamental tension in ML. Every model lives on this spectrum.

Total Error = Bias^2 + Variance + Irreducible Noise

Component	Definition	Intuition
Bias	Error from wrong assumptions in the model (underfitting)	Model is too simple to capture the true pattern
Variance	Error from sensitivity to small training data fluctuations (overfitting)	Model memorizes noise, fails on new data
Irreducible Noise	Inherent randomness in data	Cannot be reduced; sets the floor

```
High Bias (Underfit):       High Variance (Overfit):
  True relationship: ~~~~     True relationship: ~~~~
  Model fit:  ————————       Model fit:  ~^~v^~v~^~

  Too simple, misses pattern  Too complex, chases noise
```

What increases bias?

- Too simple a model (linear model for non-linear data)
- Too few features
- Too much regularization

What increases variance?

- Too complex a model (deep tree, high-degree polynomial)
- Too many features relative to data size
- Too little regularization

The Tradeoff:

- Decreasing bias usually increases variance (more complex model)
- Decreasing variance usually increases bias (simpler model)
- Goal: find the sweet spot that minimizes total error on unseen data

```source
# Illustrating bias-variance with polynomial degree
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge

for degree in [1, 3, 10, 20]:
```

```
model = Pipeline([
    ('poly', PolynomialFeatures(degree)),
    ('ridge', Ridge(alpha=1.0))
])
# degree=1: high bias (underfit)
# degree=10-20: high variance (overfit) without enough regularization
```

Interview Takeaway: When someone says "my model isn't performing well," your FIRST diagnostic questions should be: Is it underfitting (high bias) or overfitting (high variance)? These have opposite fixes.

Overfitting vs Underfitting

Symptom	Overfitting	Underfitting
Training error	Very low	High
Validation error	Much higher than training	Also high
Gap (val - train)	Large	Small
Root cause	Model too complex / too little data	Model too simple / too few features
Fix	More data, regularization, simpler model, dropout, early stopping	More features, more complex model, less regularization
Also called	High variance	High bias

Signs in practice:

- Overfit: perfect 99% training accuracy, 65% validation accuracy
- Underfit: 55% training accuracy, 54% validation accuracy (both terrible)

Regularization

Regularization adds a penalty term to the loss function to discourage complexity, reducing overfitting.

Modified Loss:

```
Total Loss = Data Loss + λ × Complexity Penalty
```

Where λ (lambda) controls the regularization strength:

- $\lambda = 0 \rightarrow$ no regularization
- $\lambda \rightarrow \infty \rightarrow$ model pushed toward all-zero weights (underfitting)

L1 Regularization (Lasso)

```
Loss = MSE + λ × Σ|wᵢ|
```

- Penalty proportional to absolute value of weights
- **Produces sparse solutions** — drives many weights to exactly zero
- Performs implicit feature selection
- Useful when many features are irrelevant
- Non-differentiable at zero (requires subgradient methods)

L2 Regularization (Ridge)

```
Loss = MSE + λ × Σwᵢ²
```

- Penalty proportional to square of weights
- **Shrinks all weights toward zero but rarely exactly zero**
- Smooth, differentiable — works well with gradient descent
- Handles multicollinearity well (correlated features share weight)
- Preferred when most features are relevant

L1 vs L2 Comparison

Property	L1 (Lasso)	L2 (Ridge)
Penalty	‘Σ	w_i
Effect on weights	Drives to exactly 0	Shrinks toward 0 (rarely 0)
Feature selection	Yes — implicit	No
Solution shape	Sparse	Dense
Differentiability	No (at 0)	Yes
Handles correlated features	Picks one, zeros others	Shares weight among all
Best for	High-dimensional, many irrelevant features	Correlated features, all features somewhat relevant
Geometric intuition	Diamond constraint (L1 ball)	Circle constraint (L2 ball)

Elastic Net: Combines both.

```
Loss = MSE + λ₁ × Σ|wᵢ| + λ₂ × Σwᵢ²
```

Best of both: sparse AND handles correlated features.

Dropout (Neural Networks) Randomly zero out neurons during training with probability `p` (typically 0.2–0.5).

```
Training:           Inference:
  [●] [○] [●]         [●] [●] [●]
  [○] [●] [●]    →    [●] [●] [●]  (scale by 1-p or use inverted dropout)
  [●] [●] [○]         [●] [●] [●]
  ○ = dropped
```

Why it works: Forces network to learn redundant representations. Each forward pass trains a different sub-network. Ensemble of ~2^n sub-networks at inference time.

Interview Gotcha: Dropout is NOT applied at inference time. Either: (1) scale activations by `(1-p)` at inference, or (2) use **inverted dropout** — divide by `(1-p)` during training so inference is unchanged (PyTorch default).

Early Stopping Stop training when validation loss stops improving (or starts increasing).

```
Training loss: ↘↘↘↘↘↘↘↘↘↘
Val loss:      ↘↘↘↘↘↗↗↗↗↗
                     ↑
                  Stop here (best model checkpoint)
```

```source
# PyTorch-style early stopping
best_val_loss = float('inf')
patience_counter = 0
PATIENCE = 5

for epoch in range(max_epochs):
    train_one_epoch(model)
    val_loss = evaluate(model, val_loader)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        save_checkpoint(model)
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= PATIENCE:
            print(f"Early stopping at epoch {epoch}")
            break

load_checkpoint(model)  # Restore best model
```

Evaluation Metrics --- Classification

The Confusion Matrix For binary classification (Positive = class of interest, e.g., Fraud):

```
                  Predicted
                Pos    Neg
Actual  Pos  [ TP  |  FN ]   ← Row: Actual Positives
        Neg  [ FP  |  TN ]   ← Row: Actual Negatives
             ↑       ↑
         Predicted  Predicted
         Positive   Negative
```

Cell	Name	Meaning
TP	True Positive	Predicted Positive, Actually Positive
TN	True Negative	Predicted Negative, Actually Negative
FP	False Positive (Type I Error)	Predicted Positive, Actually Negative
FN	False Negative (Type II Error)	Predicted Negative, Actually Positive

Core Metrics

```
Accuracy  = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)   ← Of all predicted positive, how many are?
Recall    = TP / (TP + FN)   ← Of all actual positive, how many found?
F1 Score  = 2 × (Precision × Recall) / (Precision + Recall)
Specificity = TN / (TN + FP) ← True Negative Rate
```

Precision vs Recall Tradeoff **You cannot maximize both simultaneously** (for a fixed model). Raising the classification threshold increases precision but decreases recall.

Metric	Formula	When to Prioritize
Precision	TP/(TP+FP)	FP is costly — spam filter (don't block good email), medical screening pre-test

Metric	Formula	When to Prioritize
Recall	TP/(TP+FN)	FN is costly — cancer detection (don't miss cases), fraud detection (don't miss fraud)
F1	Harmonic mean of P & R	Neither FP nor FN dominates; balanced view
F-beta	$(1+\beta^2) \times P \times R / (\beta^2 \times P + R)$	$\beta>1$: favor recall; $\beta<1$: favor precision

Interview Classic: "Would you use precision or recall for a cancer detection model?" Answer: **Recall** — a false negative (missed cancer) is far more harmful than a false positive (unnecessary biopsy). But also: optimize Precision at high Recall using PR-AUC.

ROC-AUC vs PR-AUC **ROC Curve:** Plot TPR (Recall) vs FPR at all thresholds.

- **AUC-ROC** ranges from 0.5 (random) to 1.0 (perfect)
- Insensitive to class imbalance — useful when TN matters (fraud is rare but we care about both)

PR Curve: Plot Precision vs Recall at all thresholds.

- **AUC-PR** is more sensitive to class imbalance
- Better metric when positive class is rare and more important (e.g., fraud, rare disease)
- A random classifier on imbalanced data (1% positive) gets PR-AUC ≈ 0.01, not 0.5

```
ROC Curve          PR Curve
TPR|  /‾‾          P|‾\
   | /             R| \___
   |/              E|
   └──── FPR        └────── Recall

AUC = area under curve. Higher = better.
```

Metric	Best When
ROC-AUC	Classes roughly balanced; care about ranking ability
PR-AUC	Severe class imbalance; care about positive class performance

Log Loss (Cross-Entropy Loss)

```
Log Loss = -(1/N) × Σ [yᵢ × log(ŷᵢ) + (1 - yᵢ) × log(1 - ŷᵢ)]
```

- Penalizes **confident wrong predictions** heavily ($\log(0) \to \infty$)
- Perfect for probabilistic outputs (should be calibrated probabilities, not raw scores)
- Used directly as the training objective for logistic regression and most classifiers
- Lower is better

Interview Takeaway: Log loss penalizes arrogant wrong predictions. A model that says "I'm 99% sure this is spam" and it's not spam gets a massive penalty.

Evaluation Metrics --- Regression

Metric	Formula	Interpretation	Sensitive to Outliers?
MSE	`(1/n)Σ(yᵢ - ŷᵢ)²`	Average squared error; in squared units	Very (squares errors)
RMSE	`√MSE`	Same units as target; penalizes large errors	Very
MAE	‘(1/n)Σ	$y_i - \hat{y}_i$	‘

Metric	Formula	Interpretation	Sensitive to Outliers?
R^2	`1 - SS_res/SS_tot`	Fraction of variance explained; 0–1 (higher = better)	Moderate
MAPE	‘(1/n)Σ	$(y_i - \hat{y}_i)/y_i$	‘

```source
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

mse  = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
mae  = mean_absolute_error(y_true, y_pred)
r2   = r2_score(y_true, y_pred)
```

R^2 Intuition:

- R^2 = 1.0: model perfectly explains variance
- R^2 = 0.0: model is as good as predicting the mean
- R^2 < 0: model is worse than predicting the mean

Feature Engineering

The process of transforming raw data into informative representations that improve model performance.

Types:

1. **Feature Extraction** — create new features from raw data (e.g., extract ”hour of day” from timestamp, TF-IDF from raw text)
2. **Feature Transformation** — change representation (log transform skewed variables, one-hot encode categories)
3. **Feature Interaction** — combine features (product of two variables captures non-linear relationship)
4. **Feature Aggregation** — summarize over groups (user's average spend in past 30 days)
5. **Feature Selection** — remove redundant/irrelevant features (variance threshold, mutual information, LASSO)

```source
import pandas as pd
import numpy as np

# Timestamp features
df['hour'] = pd.to_datetime(df['timestamp']).dt.hour
df['day_of_week'] = pd.to_datetime(df['timestamp']).dt.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

# Log transform (handle skewed distributions)
df['log_amount'] = np.log1p(df['transaction_amount'])

# Interaction features
df['amount_x_hour'] = df['log_amount'] * df['hour']

# Aggregation feature
df['user_avg_spend_30d'] = df.groupby('user_id')['amount'].transform(
    lambda x: x.rolling(30, min_periods=1).mean()
)
```

Feature Scaling / Normalization

Why needed: Many algorithms (SVM, KNN, gradient descent-based) are sensitive to feature magnitude.

Method	Formula	When to Use	Notes
Min-Max Scaling	`(x - min)/(max - min)`	Bounded range needed [0,1]	Sensitive to outliers
Standardization (Z-score)	`(x - μ)/σ`	Most cases; gradient descent	Assumes roughly Gaussian; not bounded
Robust Scaling	`(x - median)/IQR`	Data has outliers	Uses median and IQR instead of mean/std
Log Transform	`log(1 + x)`	Right-skewed data (income, counts)	Brings outliers closer; requires $x \geq 0$
L2 Normalization	‘x /		x

```source
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit on train, transform
X_val_scaled   = scaler.transform(X_val)         # ONLY transform, don't re-fit!
X_test_scaled  = scaler.transform(X_test)        # Same

# Common mistake: fitting scaler on all data (leakage)
# WRONG: scaler.fit(X_all); then split
```

Interview Gotcha: Fit the scaler on training data only, then apply (transform) to val and test. Fitting on all data leaks test distribution statistics into training.

Algorithms that DO need scaling: Linear/Logistic Regression, SVM, KNN, PCA, Neural Networks. **Algorithms that DON'T need scaling:** Decision Trees, Random Forests, Gradient Boosting (tree-based methods use splits, not distances).

Data Leakage

The silent killer of ML models. A model appears to perform amazingly but fails completely in production.

Definition: Information from outside the training time-window leaks into the training features, giving the model knowledge it couldn't have in real deployment.

Types:

1. **Target Leakage** — a feature that is a proxy for or contains the target
   - Example: In a loan default model, including ”bank account closed” as a feature — accounts get closed AFTER default happens
   - The feature is available at training time but won't be available at prediction time
2. **Train-Test Contamination** — test data statistics contaminate training
   - Fitting a scaler/imputer on all data before splitting
   - Using the test set for model selection decisions
   - Computing feature statistics (mean for imputation) on combined data
3. **Temporal Leakage** — using future data to predict the past
   - Using features computed from data after the prediction timestamp
   - Example: predicting Q1 sales using Q2 return rate

```source
# Leakage Example — WRONG
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)          # Uses all data including test
X_train, X_test = train_test_split(X_scaled)  # Leakage already happened!

# CORRECT
X_train, X_test = train_test_split(X)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)     # Fit only on train
X_test = scaler.transform(X_test)           # Apply to test
```

How to detect leakage:

- Suspiciously high performance (>99% accuracy on hard problem)
- Feature importance shows unexpected features dominating
- Performance collapses in production

Class Imbalance

Problem: Binary classification where one class is rare (e.g., 1% fraud, 99% legitimate).

A naive model that always predicts ”legitimate” gets 99% accuracy — utterly useless. The model learns to ignore the minority class.

Technique	How It Works	Pros	Cons
Oversampling (SMOTE)	Generate synthetic minority samples	Increases minority class signal	Can introduce noise if features overlap
Undersampling	Remove majority class samples randomly	Faster training	Loses majority class information
Class Weights	Penalize misclassifying minority class more	Simple, no data modification	May destabilize training
Threshold Moving	Lower decision threshold from 0.5	Easy post-hoc	Requires calibrated probabilities
Ensemble (BalancedRF)	Bootstrap with balanced classes per tree	Robust	More compute

```
>_ source
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

# Class weights (simple, often best first try)
weights = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weight_dict = {0: weights[0], 1: weights[1]}

# In sklearn:
model = LogisticRegression(class_weight='balanced')

# In PyTorch:
loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([99.0]))  # 99:1 imbalance

# SMOTE (synthetic oversampling)
from imblearn.over_sampling import SMOTE
X_resampled, y_resampled = SMOTE(random_state=42).fit_resample(X_train, y_train)
```

Key Insight: SMOTE: creates synthetic minority samples by interpolating between existing minority samples in feature space. Works well when classes are separable; can fail when they overlap.

Interview Takeaway: Always mention changing the evaluation metric first (use PR-AUC, F1 instead of accuracy). Then data techniques. Then loss function adjustments.

Gradient Descent

The workhorse optimization algorithm for training ML models with differentiable loss functions.

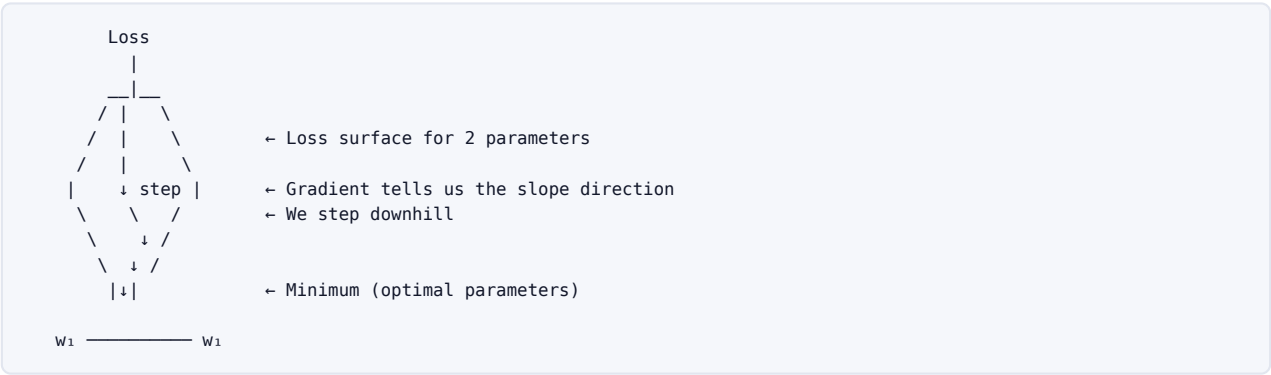
The Core Idea: Start at some parameter values, compute how much the loss would decrease if we nudge each parameter, then nudge them in that direction.

```
θ_new = θ_old - α × ∇_θ L(θ)
```

Where:

- θ = parameters (weights)
- α = learning rate (step size)
- ∇_θ L = gradient of loss with respect to parameters

The Loss Bowl Intuition:



Learning Rate:

- Too large: diverges (bounces around, overshoots minimum)
- Too small: converges too slowly (impractical)
- Just right: converges to minimum efficiently

Batch vs SGD vs Mini-Batch

Variant	Batch Size	Update Frequency	Pros	Cons
Batch GD	All N samples	Once per epoch	Stable gradient	Slow, memory intensive, stuck in saddle points
Stochastic GD (SGD)	1 sample	N times per epoch	Fast updates, escapes local minima	Noisy, oscillates
Mini-Batch GD	32–512 samples	N/batch times	Best of both: stable + fast	Batch size is a hyperparameter

Why Mini-Batch GD wins in practice:

1. Parallelizable on GPUs (matrix operations over a batch)
2. Noisiness actually helps escape sharp local minima (implicit regularization)
3. Batch Normalization works better with mini-batches

```source
# Mini-batch gradient descent
BATCH_SIZE = 128
LEARNING_RATE = 0.001

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

for epoch in range(NUM_EPOCHS):
    for X_batch, y_batch in DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True):
        optimizer.zero_grad()
        y_pred = model(X_batch)
        loss = criterion(y_pred, y_batch)
        loss.backward()     # Compute gradients via backprop
        optimizer.step()    # Update: θ = θ - α∇L
```

Advanced Optimizers:

- **Momentum:** Accumulates velocity in directions of consistent gradient, dampens oscillations
- **RMSProp:** Adaptive per-parameter learning rates (divides by running average of gradient^2)
- **Adam:** Momentum + RMSProp. Usually default choice. `lr=3e-4` (Karpathy's constant)
- **AdamW:** Adam + decoupled L2 weight decay. Standard for transformers.

The Curse of Dimensionality

As the number of features (dimensions) grows:

1. **Data becomes sparse:** In d dimensions, the same amount of data covers exponentially less of the space. With 100 training points in 2D you have decent coverage. In 100D, those 100 points are tiny specks in a vast space.
2. **Distance metrics break down:** All points become approximately equidistant. KNN, SVMs, and any distance-based algorithm suffers.
3. **Overfitting risk explodes:** More parameters = more capacity to memorize training data.
4. **Computational cost:** Many algorithms scale exponentially with dimensions.

```
1D space [0,1]: 10 samples covers it well
2D space [0,1]²: need 100 samples for same coverage
10D space [0,1]^10: need 10^10 = 10 billion samples!
```

Solutions:

- Dimensionality reduction: PCA, UMAP, t-SNE, Autoencoders
- Feature selection: Remove low-variance, low-importance features
- More data
- Regularization (implicit dimensionality control)
- Domain knowledge to identify informative features

Generalization

The central goal of ML: perform well on unseen data, not just training data.

Generalization Error = Expected error on new samples from the same distribution

Key insights:

- Training error is always a lower bound on test error (unless data is infinite)
- The gap (test error - train error) is the generalization gap
- Model capacity + data size determine generalization (VC theory, bias-variance)

What helps generalization:

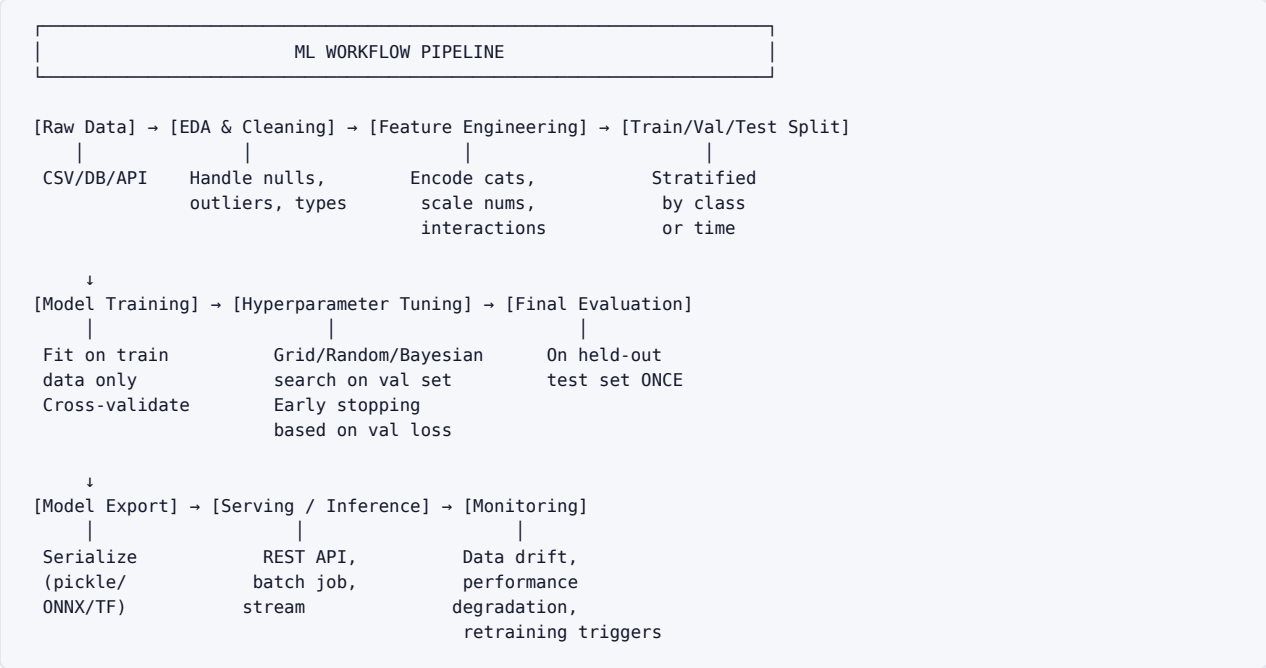
- More diverse training data

- Regularization
- Data augmentation
- Cross-validation
- Simpler models (Occam's Razor: prefer simpler explanations)
- Ensemble methods

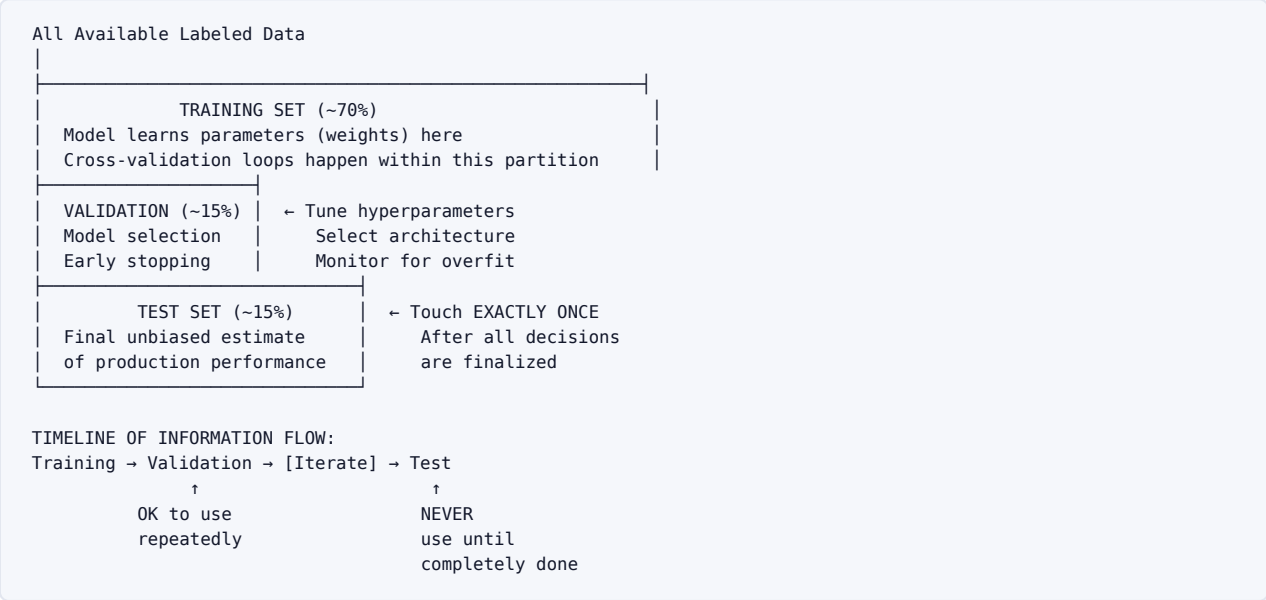
Interview Takeaway: A model with 100% training accuracy and 60% test accuracy has ZERO generalization — it's memorized the training set. The job is always to minimize generalization error, not training error.

1.5 Architecture / Diagrams

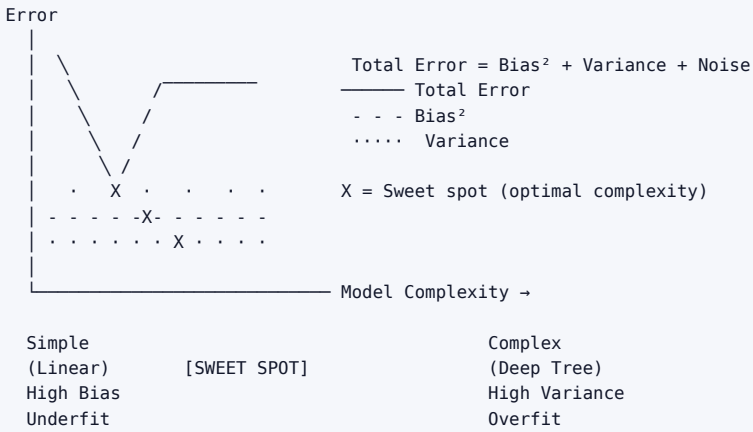
The Full ML Workflow Pipeline



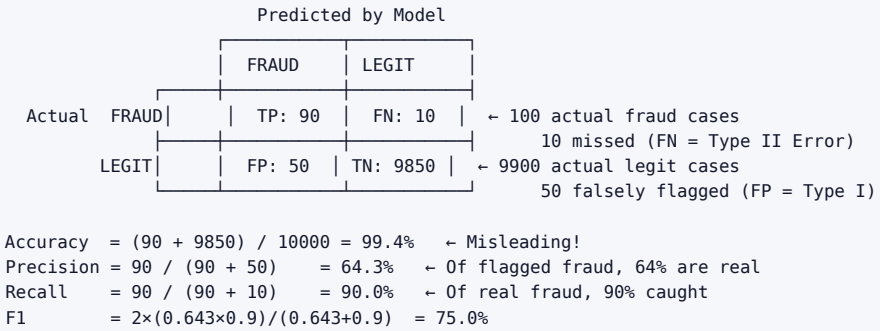
Train / Validation / Test Split Diagram



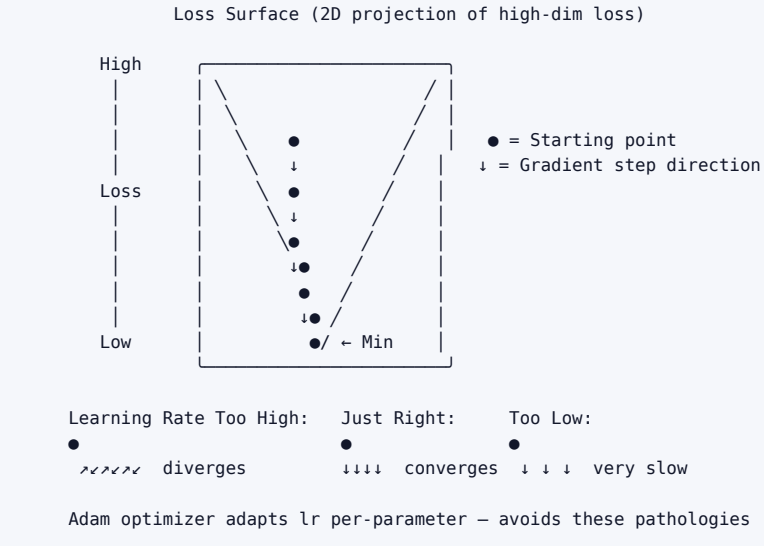
Bias--Variance Curve



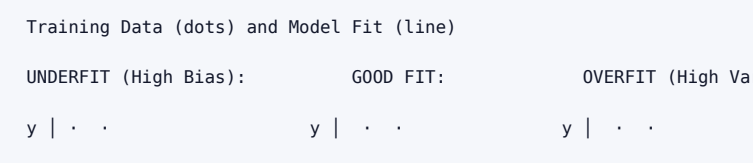
Confusion Matrix (Fraud Detection Example)

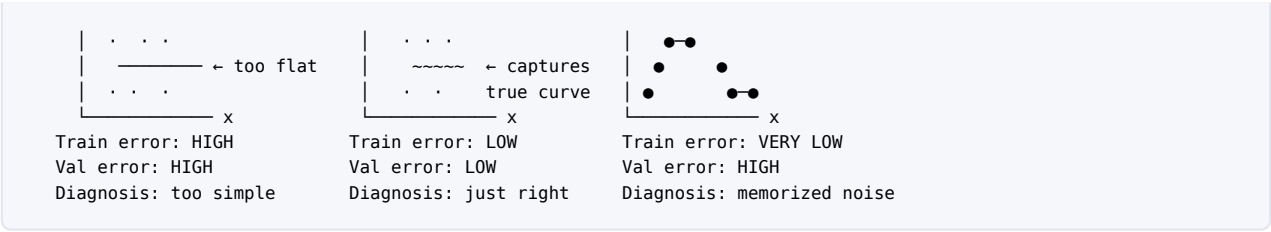


Gradient Descent on Loss Bowl



Overfitting / Underfitting Fit Curves





1.6 Real-World Examples

Google Search --- Learning to Rank

- **Task:** Given query ”best running shoes”, rank 10B+ pages by relevance
- **ML Approach:** Learning-to-rank model (RankNet, LambdaMART) on billions of (query, page, click) triples
- **Features:** PageRank, BERT query-document similarity, user engagement signals, freshness
- **Label:** Human relevance ratings (1–5) + implicit click signals
- **Key challenge:** Non-stationary distribution — queries and content evolve daily
- **Bias-variance relevance:** Simple linear ranker (high bias, misses interaction effects) vs. deep LambdaMART (risk of overfit to stale click patterns)

Netflix Recommendations --- Collaborative Filtering

- **Task:** Predict rating user u gives to movie m (to recommend unseen movies)
- **ML Approach:** Matrix Factorization (SVD, ALS) → embed users and movies in latent space
- **Label:** Explicit ratings (1–5 stars) + implicit signals (time watched, rewatch, search)
- **Key challenge:** Cold start problem (new user, new movie has no data), class imbalance (most (user, movie) pairs are unobserved)
- **Feature Engineering:** Time of day user watches, device type, country, genre embeddings

Amazon Fraud Detection --- Anomaly + Supervised

- **Task:** Flag fraudulent transactions in real-time (< 50ms latency)
- **ML Approach:** Gradient Boosted Trees (XGBoost) for known fraud patterns + isolation forest for novel anomalies
- **Features:** Transaction amount, merchant category, time since last transaction, velocity (10 transactions in 1 minute), location delta
- **Key challenge:** Extreme class imbalance (< 0.1% fraud), adversarial distribution shift (fraudsters adapt)
- **Evaluation:** PR-AUC not ROC-AUC (highly imbalanced)

Uber ETA Prediction --- Regression

- **Task:** Predict trip duration in seconds for pricing and driver dispatch
- **ML Approach:** GBM (XGBoost/LightGBM) with traffic graph features + deep learning for complex route interactions
- **Features:** Distance, time-of-day, day-of-week, historical traffic by segment, weather, events
- **Evaluation Metric:** MAPE (Mean Absolute Percentage Error) — because a 5-min error on a 10-min trip is worse than on a 60-min trip
- **Real-world concern:** Training data from completed trips (selection bias — trips that didn't complete aren't in training)

1.7 Real-Life Analogies --- Training a Student Through School

Every ML concept maps perfectly to a student's academic journey — once you see this, you'll never forget the terminology.

ML Concept	School Analogy	One-Line Explanation
Data (training set)	Textbooks and practice problems	The raw material the student learns from
Model	The student's brain	The learnable system that stores patterns
Parameters/weights	The student's understanding and heuristics	Adjustable internal representations
Training	Studying and doing homework	Iteratively adjusting understanding to minimize errors
Loss function	Number of mistakes on practice problems	The signal that tells us how wrong we are
Gradient descent	A tutor pointing out WHY answers are wrong and nudging the student to correct their thinking	Step-by-step guided improvement toward fewer mistakes
Learning rate	How big a conceptual leap the student takes per study session	Too big = confused; too small = no progress
Validation set	Mock exams / practice tests	Unseen problems during studying — gauge true understanding
Test set	The final exam	Seen only once, after ALL studying is done
Overfitting	Memorizing past paper answers without understanding	Scores 100% on known papers, fails when questions change
Underfitting	Not studying enough, oversimplifying	Fails both practice tests AND the final
Regularization	Forcing the student to explain principles, not just recall	Discourages rote memorization, forces deeper understanding
High bias	Student believes all history is just dates (oversimplified mental model)	Wrong assumptions → systematic errors regardless of data
High variance	Student panics on any unfamiliar question phrasing	Too sensitive to surface details, fails to generalize
Feature engineering	Highlighting the most important passages, making summary cards	Transforming raw input into informative representations
Feature scaling	Converting all measurements to the same units	Ensures no single feature dominates due to scale
Dropout	Randomly covering parts of notes while reviewing	Forces redundant learning paths, prevents over-reliance on any single fact
Early stopping	Stopping study when practice test scores stop improving	Avoid over-studying a practice paper to the point of memorizing it
Data leakage	Student sees the final exam questions before sitting it	Invalidates the performance estimate — cheating
Class imbalance	99 easy questions and 1 trick question in practice — student learns to ignore trick questions	Minority class gets ignored because it barely affects overall score
Cross-validation	Taking 5 different mock exams, averaging scores	More robust performance estimate than a single mock
Generalization	Being able to apply learned concepts to brand-new problems in life	The ultimate goal — performance on unseen data
Irreducible noise	Ambiguous exam questions that even perfect studying can't answer	Inherent randomness; cannot be eliminated

1.8 Memory Tricks / Mnemonics

Precision vs Recall

”Precise people don't waste your time (no FP). Recall people find everyone (no FN).”

- **P**recision → Predicted Positives → avoid **P**estering innocents (FP)
- **R**ecall → Real Positives → **R**escue everyone (avoid FN)

Bias vs Variance

"Bias is a BAD ASSUMPTION. Variance is WILD SENSITIVITY."

- **B**ias = **B**ad assumption before seeing data (too simple)
- **V**ariance = Very sensitive to which training samples you use (too complex)

L1 vs L2 Regularization

"L1 = Lasso (lasso ropes in/eliminates features). L2 = Ridge (ridge shrinks but keeps all)."

- **L1 Lasso → Lean (sparse, zeros out features)**
- **L2 Ridge → Rounded (all weights survive, just smaller)**

Type I vs Type II Error

"I cried wolf (Type I = False Positive). II missed the wolf (Type II = False Negative)."

- Type **I** Error → False Positive → accused an innocent
- Type **II** Error → False Negative → missed the real criminal

Confusion Matrix: TN, FP, FN, TP (reading order)

"Truly the most Frequently misunderstood Perfectly — True Negative, False Positive, False Negative, True Positive"

- First word (True/False) = Was the prediction correct?
- Second word (Positive/Negative) = What did we predict?

Cross-Validation

"K-Fold = K chances to be the test set"

- 5-fold = each fold gets 1 turn as validation set
- Best estimate when data is scarce

Gradient Descent Learning Rate

"Goldilocks Rate: not too hot, not too cold"

- Too high: explodes / oscillates
- Too low: glacially slow
- Just right: smooth convergence

Overfitting vs Underfitting

"Overfit = Over-smart student who studied only past papers. Underfit = Under-prepared student who barely studied."

The Bias-Variance-Noise Decomposition

MSE = Bias^2 + Variance + Noise — "Every Model Succeeds by Balancing Variance and Noise"

1.9 Common Interview Questions

Q1: What is the bias-variance tradeoff?

Model Answer: Bias-variance tradeoff is the tension between two sources of error that prevent ML models from generalizing. Total test error decomposes as: **Bias^2 + Variance + Irreducible Noise**.

- **Bias** is error from overly simplistic assumptions. A linear model fit to quadratic data has high bias — it systematically underpredicts in some regions regardless of how much data you provide.
- **Variance** is error from excessive sensitivity to training data fluctuations. A depth-20 decision tree might perfectly fit the training set but produce wildly different predictions on slight data variations — it's memorizing noise.

The tradeoff: making a model more complex reduces bias but increases variance. Finding the sweet spot is the core challenge of model selection.

Diagnosis and Fix:

- High bias → training error AND val error are both high → use more complex model, add features, reduce regularization
- High variance → training error is low but val error is much higher → add regularization, more data, simpler model, dropout

Follow-up: *"How do you know which problem you have?"* Plot learning curves: training loss and validation loss vs. training set size. High bias → both losses plateau high. High variance → large gap between training loss (low) and validation loss (high).

Q2: Explain precision and recall. When would you use each?

Model Answer: Both measure different aspects of a classifier's performance on positive class:

- **Precision = TP/(TP+FP):** Of all cases the model flagged as positive, what fraction actually is? Measures how trustworthy positive predictions are.
- **Recall = TP/(TP+FN):** Of all actual positives, what fraction did the model catch? Measures how comprehensive the model is.

They trade off against each other via the decision threshold. Lowering the threshold catches more positives (higher recall, lower precision). Raising it makes predictions more conservative (higher precision, lower recall).

When to use which:

- **Prioritize Recall** when FN is costly: cancer detection (missing a cancer = patient dies), fraud detection (missing fraud = financial loss), content moderation (missing CSAM = harm to children)
- **Prioritize Precision** when FP is costly: email spam filter (blocking legitimate email = user frustration), recommendation systems (showing irrelevant results = user churn)
- **Use F1** when neither dominates; **use PR-AUC** when you need a threshold-independent metric with class imbalance

Follow-up: *"What's the F1 score and why harmonic mean?"* F1 = 2PR/(P+R). Harmonic mean penalizes extreme imbalance — a model with Precision=1.0, Recall=0.01 gets F1=0.02, not 0.5 (arithmetic mean). It forces both to be reasonably high.

Q3: What is data leakage and how do you detect it?

Model Answer: Data leakage occurs when information from outside the training time-window or from the target variable flows into the training features, making a model appear to perform better than it will in production.

Three common forms:

1. **Target leakage:** Using a feature that is causally downstream of the target (e.g., "refund requested" to predict "product will be returned" — they happen simultaneously or the feature is a consequence)
2. **Train-test contamination:** Fitting preprocessing (scalers, imputers, TF-IDF vocabulary) on all data before splitting, so test statistics leak into training
3. **Temporal leakage:** Using future information to predict the past (e.g., using month-end aggregate to predict daily events)

Detection signals:

- Suspiciously high performance (>98% on a hard problem)
- A feature that shouldn't predict the target has very high feature importance
- Large performance drop when model goes to production

Prevention:

- Always split first, then fit any preprocessing on training data only
- For time-series: use strict temporal splits (train on past, val on future)
- Scrutinize feature definitions: "would this value be available at prediction time?"

Follow-up: *"How do you handle time-series specifically?"* Use TimeSeriesSplit from sklearn which always ensures training window is before validation window. Never shuffle time-series data.

Q4: Your model has 98% accuracy but the business is unhappy. What's wrong?

Model Answer: Classic class imbalance problem. If the dataset is 98% negative class, a model that always predicts "negative" achieves 98% accuracy with zero utility.

Diagnostic steps:

1. Check class distribution — is one class « 1%?
2. Look at confusion matrix — are all FN and FP rates acceptable?
3. Compute F1, Precision, Recall, PR-AUC — these expose class imbalance issues

Solutions (in order of simplicity):

1. Change metric → use PR-AUC or F1 for model selection
2. Adjust decision threshold → find threshold maximizing F1 or recall@fixed-precision
3. Class weights → `class_weight='balanced'` in sklearn
4. Resampling → SMOTE for oversampling minority, or random undersampling majority
5. Use different algorithm → Isolation Forest, anomaly detection methods

Follow-up: *"What if SMOTE makes things worse?"* SMOTE fails when classes heavily overlap in feature space — synthetic minority samples get created in ambiguous regions. Try cost-sensitive learning (class weights) or ensemble methods (BalancedRandomForest) instead.

Q5: Explain the difference between L1 and L2 regularization.

Model Answer: Both add a penalty to the loss function to prevent overfitting:

- **L1 (Lasso):** penalty = $\lambda\Sigma|w_i|$ → produces **sparse solutions** (many weights exactly zero)
- **L2 (Ridge):** penalty = $\lambda\Sigma w_i^2$ → **shrinks all weights** toward zero (rarely exactly zero)

Geometric intuition: L1 penalty corresponds to a diamond-shaped constraint region. The loss function contours typically touch the diamond at a corner (where most weights = 0) → sparsity. L2 penalty is a circular constraint. The smooth circular boundary means optimal points rarely align with axes → no exact zeros.

Practical choice:

- Use L1 when you have many features and suspect most are irrelevant — it performs feature selection
- Use L2 when features are correlated or all somewhat relevant — it distributes weight evenly
- Use Elastic Net (both) when you want feature selection + handling correlated features

Follow-up: *"How does the lambda hyperparameter affect training?"* Higher λ → more regularization → simpler model → moves along tradeoff toward higher bias/lower variance. Tune via cross-validation; too high → underfit; too low → overfit.

Q6: What is cross-validation and when do you use it?

Model Answer: Cross-validation is a resampling technique to estimate model performance on unseen data and tune hyperparameters when data is limited.

K-Fold CV: Divide training data into K folds. Train on K-1 folds, validate on 1 fold. Rotate K times. Final score = mean ± std of K scores.

When to use:

- Dataset is small and a single train/val split gives high-variance estimates
- Comparing multiple models/hyperparameter settings with statistical rigor
- No separate test set exists (though ideally you still hold one out)

Variants for specific situations:

- **Stratified K-Fold:** Classification with class imbalance — maintains class ratio in each fold
- **Time-Series Split:** Temporal data — train window always precedes validation window
- **Nested CV:** Proper hyperparameter tuning without test set contamination

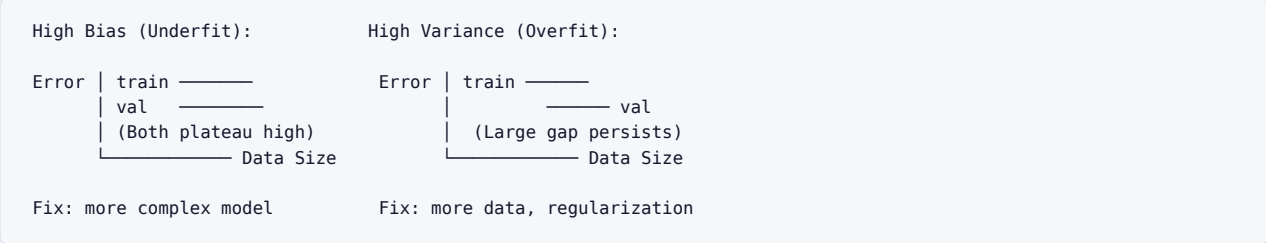
Key point: CV is used for model selection and performance estimation, never for final evaluation. The test set remains untouched. The CV is entirely within the training partition.

Follow-up: *"What's the difference between 5-fold and LOO CV?"* LOO (Leave-One-Out): each sample is the validation set once. Gives nearly unbiased estimate but O(N) training runs. For N=10,000 that's 10,000 model fits — computationally expensive. 5-fold is a practical compromise.

1.10 Senior-Level Discussion Points

1. The Learning Curve as a Diagnostic Tool

Beyond just looking at final metrics, senior engineers analyze learning curves — performance vs. training set size:



If error plateaus early regardless of more data → high bias. Adding data won't help; need better model or features. **If gap between train/val stays large with more data → high variance.** Need regularization or architecture changes.

2. The No Free Lunch Theorem

There is no single algorithm that works best across all problems. Every algorithm makes assumptions. Linear regression assumes linear relationships; KNN assumes local smoothness; Naive Bayes assumes feature independence. Choosing algorithms is always about matching inductive biases to data structure.

Practical implication: always try simple baselines (logistic regression, linear regression, majority class) before complex models. If a deep network doesn't beat logistic regression significantly, the problem might be inherently linear.

3. Statistical Significance of Evaluation Results

A model scoring 0.85 AUC vs. 0.84 AUC — is that meaningful? Senior engineers run paired t-tests or McNemar's test to determine if differences are statistically significant. With cross-validation, compute mean ± confidence interval. Never declare a winner based on a single train-test split.

4. Calibration vs. Discrimination

Discrimination: Does the model rank positive examples higher? → ROC-AUC measures this. **Calibration:** When the model says P(fraud) = 0.7, is the actual fraud rate among such predictions ≈ 70%? → Brier score, calibration curve measure this.

Many high-AUC models are poorly calibrated. For downstream decision-making (setting fraud thresholds, resource allocation), calibration is critical. Platt scaling or isotonic regression can post-hoc calibrate a model.

5. Distributional Shift --- The Biggest Production Failure Mode

Models trained on historical data fail when:

- **Covariate shift:** P(X) changes but P(Y|X) stays same (user demographics shift; feature distribution changes)
- **Label shift:** P(Y) changes but P(X|Y) stays same (more fraud during holidays)

- **Concept drift:** P(Y|X) itself changes (fraud tactics evolve; language meaning changes)

Detection: monitor input feature distributions (KL divergence, PSI), model output distributions, and actual labels if available. Mitigation: regular retraining, importance weighting, domain adaptation.

6. Inductive Bias and Regularization as Two Sides of a Coin

Regularization is how we impose prior knowledge into model training. L2 regularization says ”weights should be small” (a prior that says effects are generally moderate). L1 says ”most features are irrelevant.” Dropout says ”no single neuron should be critical.” These are all forms of Bayesian priors — L2 corresponds to a Gaussian prior on weights, L1 to a Laplace prior. Senior engineers understand regularization from both frequentist and Bayesian perspectives.

7. Hyperparameter Optimization at Scale

- **Grid Search:** Exhaustive; works well for 1–2 hyperparameters. O(n^k) complexity — explodes.
- **Random Search:** Sample randomly; surprisingly effective (most hyperparameters are insensitive). Smoother coverage of important ones.
- **Bayesian Optimization (Optuna, HyperOpt):** Build surrogate model of performance landscape; sample intelligently to find optimum with fewer evaluations. Better for expensive training runs.
- **Population-Based Training (PBT):** Used at DeepMind; train a population of models, periodically copy best model's weights to poor performers with mutated hyperparameters.

1.11 Typical Mistakes Candidates Make

1. Using Test Set for Validation

Mistake: ”I tuned my hyperparameters and kept testing on the test set until I found the best.”
Why it's wrong: You've now overfit to the test set. It's no longer an unbiased estimate. Report results will be optimistic; production will disappoint.
Correct: Use a separate validation set OR nested cross-validation for all tuning decisions.

2. Ignoring Class Imbalance

Mistake: ”My model gets 99% accuracy, so it's great!”
Why it's wrong: On a 99:1 imbalanced dataset, always predicting the majority class achieves 99%. Precision/recall on the minority class is 0%.
Correct: Always check class distribution first. Use PR-AUC, F1, or per-class metrics.

3. Not Scaling Features Before Distance-Based Algorithms

Mistake: Running KNN or SVM on features with wildly different scales (age in [0,100] vs. income in [0,1,000,000]).
Why it's wrong: Distance is dominated by the large-scale feature; other features become irrelevant.
Correct: Always standardize or normalize features before KNN, SVM, logistic regression, neural networks.

4. Fitting Preprocessors on All Data

Mistake: `scaler.fit(X_all); X_train, X_test = split(X_all_scaled)`
Why it's wrong: Data leakage — test set statistics (mean, std) contaminate scaler, which then informs training.
Correct: Split first, then fit preprocessors on training data only.

5. Confusing Correlation with Causation in Features

Mistake: Including ”number of hospital visits” as a feature to predict ”health outcome” — but hospital visits are caused by poor health, not the other way.
Why it's wrong: Feature may be a proxy for the label, causing leakage. Model may also be useless in production if the feature isn't available.

6. Not Checking for Temporal Leakage

Mistake: Shuffling time-series data before splitting into train/test.
Why it's wrong: Test samples from time T may appear in training if data is shuffled. Model "sees the future."
Correct: Always sort by time; train on past, validate on future.

7. Confusing Parameters vs. Hyperparameters

Mistake: "I tuned the number of training epochs using gradient descent."
Why it's wrong: Epochs is a hyperparameter — it's set before training and cannot be optimized by gradient descent.
Correct: Parameters (weights) are learned via gradient descent. Hyperparameters (lr, layers, regularization strength) are tuned via cross-validation or search.

8. Misinterpreting R^2

Mistake: "$R^2 = 0.8$ means we explain 80% of variance, so the model is good."
Why it's wrong: R^2 is relative to the mean baseline and sensitive to outliers. A model can have high R^2 but terrible RMSE if the baseline variance is high. Also, R^2 can be negative for poor models on test sets.

9. Reporting Only a Single Metric

Mistake: "The model achieves 0.93 F1."
Why it's wrong: F1 doesn't tell you about calibration, runtime, interpretability, or whether the performance is stable across subgroups.
Correct: Report a suite: accuracy, F1, PR-AUC, calibration curve. Disaggregate by important subgroups.

10. Treating Gradient Descent Convergence as Guaranteed

Mistake: "Just run gradient descent long enough and it'll find the optimal solution."
Why it's wrong: For non-convex loss surfaces (neural networks), gradient descent finds a local minimum or saddle point, not necessarily a global minimum. Learning rate, initialization, and batch size all affect which solution is reached.

1.12 How This Connects to Other Topics

Deep Learning

ML fundamentals are the foundation:

- **Bias-variance:** Neural networks can represent any function but overfit without regularization (dropout, weight decay, data augmentation, BatchNorm)
- **Gradient descent:** Backpropagation is just automatic differentiation of the chain rule, feeding into mini-batch SGD
- **Regularization:** Dropout, L2 weight decay, BatchNorm (implicit regularization), data augmentation are all regularization forms
- **Evaluation metrics:** Same metrics apply; deep learning adds perplexity (language), FID (image generation), BLEU (translation)
- **The loss surface:** Neural network losses are non-convex; modern optimizers (Adam, LARS, Shampoo) navigate these

MLOps

- **Train/val/test discipline:** MLOps formalizes this in pipeline code — DVC, MLflow track dataset versions and splits
- **Data leakage:** Feature stores (Feast, Tecton) ensure consistent feature computation between training and serving
- **Monitoring:** Data drift detection (Evidently, Arize) catches covariate shift; retraining pipelines respond to performance degradation

- **Evaluation metrics:** A/B testing in production validates offline metric improvements translate to online gains

System Design

ML interviews often combine system design:

- **Feature engineering at scale:** How do you compute user's 30-day purchase history for 100M users in real-time? → Feature store, precomputed materialized views
- **Class imbalance at scale:** Negative sampling for embedding training (word2vec, YouTube DNN), under-sampling for training efficiency
- **Gradient descent at scale:** Distributed training (data parallelism, model parallelism, gradient compression) for billion-parameter models
- **Evaluation at scale:** Online experiments (A/B tests), metric pipelines, experiment tracking

Statistics

ML fundamentals ARE applied statistics:

- **Bias-variance tradeoff:** Relates to statistical estimation theory (MSE decomposition)
- **Cross-validation:** Bootstrap estimation, hypothesis testing for model comparison
- **Regularization:** Equivalent to maximum a posteriori (MAP) estimation with priors (L2 = Gaussian prior, L1 = Laplace prior)
- **Log loss:** Cross-entropy; MLE of Bernoulli distribution parameters
- **Evaluation:** Statistical significance testing (McNemar's test, DeLong's AUC comparison), confidence intervals

1.13 FAANG Interview Tips

Before the Interview

1. **Know the vocabulary cold:** Bias, variance, overfitting, regularization, precision, recall, AUC, cross-validation. These come up in every ML interview, often as warm-ups. Fluency signals credibility.
2. **Practice the loss decomposition:** Write out `Total Error = Bias² + Variance + Noise` and explain each term. This shows you think in first principles.
3. **Prepare concrete examples:** Have 2–3 real scenarios where you applied these concepts. ”I noticed our model had high variance when I saw the train-val gap was 20% AUC — I added L2 regularization and the gap dropped to 3%.”

During the Interview

4. **State assumptions and tradeoffs:** Never say ”I would use X.” Say ”I would try X because [reason], but it has [limitation]. If [condition], I'd consider Y instead.” Interviewers are evaluating your judgment, not just your knowledge.
5. **Drive toward metrics early:** ”What metric are we optimizing?” and ”What's the business goal?” are always correct first questions. A model that optimizes the wrong metric is useless.
6. **Diagnose before prescribing:** When given a scenario (model doesn't perform well), ask diagnostic questions: What's the class balance? What's the train vs. val performance gap? What are the features? Interviewers want to see systematic thinking.
7. **Quantify when possible:** Don't say ”accuracy is bad for imbalanced datasets.” Say ”on a 99:1 dataset, a naive classifier achieves 99% accuracy by always predicting the majority class — F1 on the minority class is 0, which is why we use PR-AUC instead.”
8. **Connect to production:** After discussing model training, say ”In production, I'd also worry about [data drift/training-serving skew/retraining schedule].” Senior roles require production thinking.

Common Traps

9. **"What's your model doing at inference?" trap:** If you say "I use dropout," interviewers will ask "do you use dropout at inference?" The answer is no (or scale by (1-p)). Know the distinction.
10. **"What's your test set used for?" trap:** "I fine-tuned my model using test set feedback" is immediately disqualifying. The test set is sacred — one-shot use after all decisions are finalized.
11. **"Why L1 over L2?" trap:** Many candidates say "L1 is better." The correct answer is "it depends — L1 for feature selection, L2 for correlated features, Elastic Net for both."
12. **The "accuracy is great" trap:** Never lead with accuracy as the key metric without checking class balance. Saying "I achieved 99% accuracy" on an imbalanced dataset signals a red flag to interviewers.

1.14 Revision Cheat Sheet

10-Minute Summary

Machine Learning = learning a function f(x)→y from data. Four paradigms: supervised (has labels), unsupervised (discovers structure), self-supervised (auto-generates labels from data itself), reinforcement (learns from reward signals). The ML lifecycle: data → features → model → evaluate → deploy → monitor. Always split data into train/val/test — test is touched once. Cross-validation gives robust performance estimates on small datasets. The fundamental tension: bias (too simple) vs. variance (too complex) — total error = Bias^2 + Variance + Noise. Regularization (L1, L2, dropout, early stopping) reduces variance. Evaluation depends on the task: classification uses accuracy/F1/ROC-AUC/PR-AUC; regression uses RMSE/MAE/R^2. Feature engineering and scaling are critical preprocessing steps. Data leakage silently inflates performance. Class imbalance requires metric changes and resampling. Gradient descent optimizes parameters iteratively — mini-batch is standard, Adam is the default optimizer. High dimensionality hurts all distance-based methods. Generalization on unseen data is the ultimate goal.

Key Points Checklist

☐ Total Error = Bias^2 + Variance + Irreducible Noise
☐ High bias: both train and val error are high → need more complex model or features
☐ High variance: low train error, high val error → need regularization or more data
☐ L1 (Lasso) = sparse (zeros out features); L2 (Ridge) = dense (shrinks all weights)
☐ Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = harmonic mean of both
☐ Prioritize Recall when FN is costly (cancer detection); Precision when FP is costly (spam)
☐ ROC-AUC: good for balanced classes; PR-AUC: better for imbalanced classes
☐ Test set is touched EXACTLY ONCE after all decisions finalized
☐ Fit scalers/imputers on training data ONLY — never on all data
☐ Gradient descent: $\theta = \theta - \alpha\nabla L$; mini-batch is standard; Adam is default optimizer
☐ Dropout: applied during training, NOT inference (or use inverted dropout)
☐ Early stopping: restore best checkpoint when val loss stops improving
☐ SMOTE: interpolates minority samples in feature space
☐ Cross-entropy loss penalizes confident wrong predictions heavily ($\log(0) \to \infty$)
☐ R^2 = 1 - (residual variance / total variance); 0 = mean baseline; <0 = worse than mean

Cheat-Sheet Table

Concept	Formula / Key Idea	Interview Punchline
Bias	Systematic error from wrong assumptions	Model too simple; fix with more complexity
Variance	Sensitivity to training data	Model too complex; fix with regularization
Total MSE	Bias^2 + Variance + Noise	Can't reduce noise; optimize the other two
Precision	TP/(TP+FP)	Of predicted positives, fraction that are real
Recall	TP/(TP+FN)	Of real positives, fraction we caught

Concept	Formula / Key Idea	Interview Punchline
F1	2PR/(P+R)	Harmonic mean; forces both P and R to be high
ROC-AUC	Area under TPR-FPR curve	Threshold-independent ranking ability
PR-AUC	Area under Precision-Recall curve	Better than ROC-AUC for imbalanced data
L1 (Lasso)	Loss + $\lambda\Sigma$	w
L2 (Ridge)	Loss + $\lambda\Sigma w^2$	Dense weights; handles correlated features
Elastic Net	L1 + L2	Best of both worlds
Dropout	Zero neurons with prob p at train time	Ensemble of sub-networks; don't apply at test
Early Stopping	Stop when val loss stops improving	Cheapest regularization trick
SMOTE	Interpolate minority samples	Fix class imbalance without losing majority data
Gradient Descent	$\theta \leftarrow \theta$ - $\alpha\nabla$L	Step downhill on loss surface
Adam	Adaptive lr + momentum	Default optimizer; lr=3e-4 is a good start
Data Leakage	Future info in past prediction	Undetectable until production failure
Cross-Entropy	-Σy·log($\hat{y}$)	Penalizes confident errors; training loss for classifiers
R^2	1 - SS_res/SS_tot	1=perfect; 0=mean baseline; <0=worse than baseline

Most Important Concepts (Ranked by Interview Frequency)

1. **Bias-Variance Tradeoff** — asked in ~90% of ML interviews; know the diagnosis (learning curves) and the fix
2. **Precision vs Recall** — asked constantly; always ask ”what's more costly, FP or FN?”
3. **Data Leakage** — senior engineers are defined by catching this; know all three types
4. **Regularization (L1 vs L2)** — geometric intuition + practical differences
5. **Cross-validation** — when/why; stratified; time-series split
6. **Class Imbalance** — metric choice first, then techniques
7. **Gradient Descent** — mini-batch mechanics, learning rate effects, Adam
8. **Evaluation Metrics** — full suite: F1, AUC, log-loss, R^2; when to use each
9. **Train/Val/Test discipline** — sacred rule: test set is once-only
10. **Overfitting diagnosis** — learning curves, train-val gap, complexity vs. data size

Last updated: June 2026 | Next: `02-supervised-learning-algorithms.md`

2 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

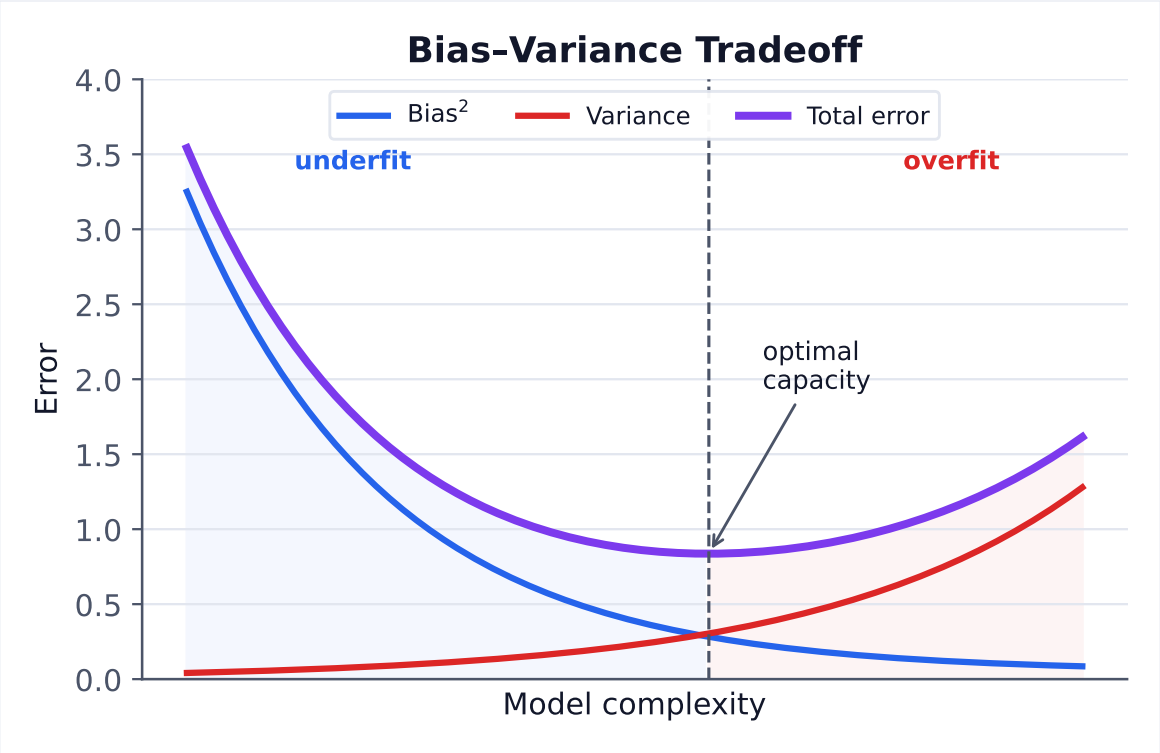


Figure 1. Total generalization error decomposes into bias^2 + variance + irreducible noise. Minimizing it means balancing model capacity, not maximizing fit.

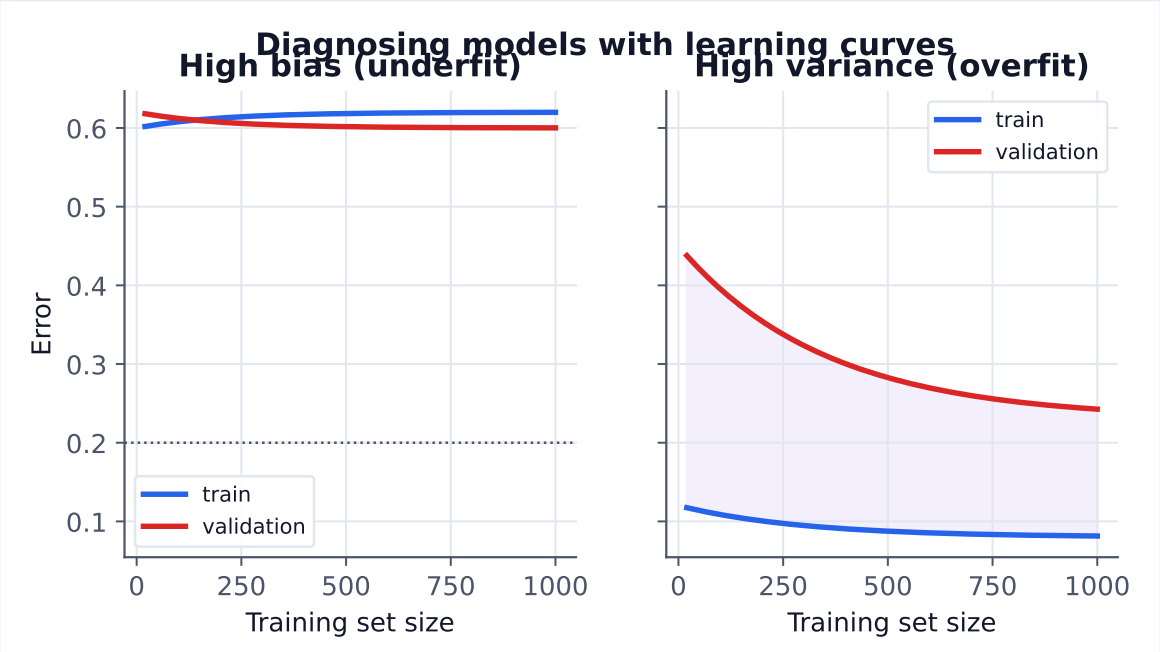


Figure 2. A large train–validation gap that closes with more data signals variance; high error on both that more data won't fix signals bias.